Визуализатор графа зависимостей Maven (вариант 30)

Консольное приложение на Python 3.8+ (только стандартная библиотека). Строит граф зависимостей пакета Maven: берёт POM-файлы из репозитория по URL (формат Maven), собирает транзитивные зависимости обходом в ширину (BFS) и выводит граф в виде текста на языке D2 и ASCII-дерева. Менеджеры пакетов и сторонние библиотеки для получения зависимостей не используются.

Запуск

```
python3 src/main.py config.json [--stage N]       # или ./run.sh config.json
```

`--stage N` (1-5) показывает вывод соответствующего этапа; по умолчанию - последний реализованный. Параметры задаются в JSON-файле (примеры - в `tests/configs/`):

Параметр	Тип	Смысл
`package`	строка	Maven: `groupId:artifactId`; тестовый режим: имя из больших латинских букв (`A`)
`version`	строка	версия пакета (обязательна для Maven)
`repository`	строка	URL репозитория (`http://`, `https://`, `file://`) или путь к файлу тестового репозитория
`test_mode`	true/false	режим работы с тестовым репозиторием (по умолчанию false)
`ascii_tree`	true/false	выводить граф в виде ASCII-дерева (по умолчанию false)
`max_depth`	целое >= 0	максимальная глубина анализа (по умолчанию 3)
`reverse`	true/false	обратные зависимости, только для тестового репозитория (по умолчанию false)

Структура

```
src/main.py       точка входа, разбор командной строки
src/config.py     чтение и проверка JSON-конфигурации
src/maven.py      загрузка и разбор POM, прямые зависимости (этап 2)
src/testrepo.py   тестовый репозиторий - файл 'A: B C' (этап 3)
src/graph.py      BFS без рекурсии, циклы, обратные зависимости, рёбра
src/render.py     ASCII-дерево и описание на языке D2 (этап 5)
src/stages.py     вывод каждого этапа
tests/            модульные тесты (unittest), скрипты демонстрации test_stageN.sh,
                  конфигурации tests/configs, данные tests/data,
                  макет Maven-репозитория tests/mock_repo
run.sh, run.bat   запуск
```

Тесты: `python3 -m unittest discover -s tests`. Демонстрации этапов: `bash tests/test_stage1.sh` и т. д.

Этап 1. Минимальный прототип с конфигурацией

- Источник параметров - JSON-файл (`src/config.py`). При запуске выводятся все параметры в формате «ключ: значение».
- Параметры: имя пакета, URL репозитория или путь к тестовому файлу, режим тестового репозитория, ASCII-дерево, максимальная глубина (плюс `version`, нужная для Maven, и `reverse` для этапа 4).
- Обработка ошибок для **всех** параметров: нет файла/каталог вместо файла/некорректный JSON/корень не объект; неизвестный параметр; отсутствующие обязательные; неверное имя пакета (для Maven и для теста); неверная версия; неверный URL или несуществующий файл репозитория; логические параметры не `true/false`; глубина не целое или отрицательная. Все найденные ошибки печатаются сразу, код возврата 2.

Демонстрация (`bash tests/test_stage1.sh`):

```
$ python3 src/main.py tests/configs/stage1_ok.json --stage 1
package: org.example:app
version: 1.0
repository: http://localhost:8765
test_mode: false
ascii_tree: true
max_depth: 2
reverse: false
[код возврата: 0]

$ python3 src/main.py tests/configs/stage1_ok_test_mode.json --stage 1
package: A
version:
repository: tests/data/graph_basic.txt
test_mode: true
ascii_tree: true
max_depth: 3
reverse: false
[код возврата: 0]

$ python3 src/main.py tests/configs/errors/ascii_not_bool.json --stage 1
Ошибка конфигурации: ascii_tree: ожидается true или false
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/bad_json.json --stage 1
Ошибка конфигурации: некорректный JSON: Expecting property name enclosed in double quotes (строка 2, столбец 1)
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/bad_package_maven.json --stage 1
Ошибка конфигурации: package: 'app' - ожидается вид groupId:artifactId
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/bad_package_test.json --stage 1
Ошибка конфигурации: package: 'ab' - в тестовом режиме имя состоит из больших латинских букв (например, A)
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/bad_url.json --stage 1
Ошибка конфигурации: repository: URL должен начинаться с http://, https:// или file://
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/bad_version.json --stage 1
Ошибка конфигурации: version: ожидается строка вида 1.0.0
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/depth_negative.json --stage 1
Ошибка конфигурации: max_depth: число не должно быть отрицательным
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/depth_string.json --stage 1
Ошибка конфигурации: max_depth: ожидается целое число
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/missing_package.json --stage 1
Ошибка конфигурации: package: параметр обязателен
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/missing_repository.json --stage 1
Ошибка конфигурации: repository: параметр обязателен
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/missing_version.json --stage 1
Ошибка конфигурации: version: параметр обязателен для Maven
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/not_object.json --stage 1
Ошибка конфигурации: корень JSON должен быть объектом {...}
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/repo_file_missing.json --stage 1
Ошибка конфигурации: repository: файл не найден: tests/data/no_such.txt
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/repo_is_dir.json --stage 1
Ошибка конфигурации: repository: это не файл: tests/data
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/reverse_in_maven.json --stage 1
Ошибка конфигурации: reverse: обратные зависимости доступны только для тестового репозитория (test_mode)
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/test_mode_not_bool.json --stage 1
Ошибка конфигурации: test_mode: ожидается true или false
[код возврата: 2]

$ python3 src/main.py tests/configs/errors/unknown_key.json --stage 1
Ошибка конфигурации: color: неизвестный параметр
Ошибка конфигурации: repository: файл не найден: x
[код возврата: 2]

$ python3 src/main.py tests/no_such.json --stage 1
Ошибка конфигурации: файл конфигурации не найден: tests/no_such.json
[код возврата: 2]

$ python3 src/main.py tests/data --stage 1
Ошибка конфигурации: это каталог, а не файл: tests/data
```

```
[код возврата: 2]
```

Этап 2. Сбор данных

- Формат пакетов - Java (Maven). Модуль `src/maven.py`: по координатам `groupId:artifactId:version` строится URL POM-файла по раскладке Maven-репозитория (`<repo>/<group/как/путь>/<artifact>/<version>/<artifact>-<version>.pom`), файл скачивается через `urllib`, XML разбирается через `xml.etree` (менеджеры пакетов и сторонние библиотеки не используются).
- Прямые зависимости берутся из `<dependencies>`. Учитывается: наследование от родительского POM (`<parent>`: группа, версия, свойства, `dependencyManagement`), подстановка `${свойств}` (`${project.version}` и своих), версии из `dependencyManagement`.
- В граф попадают зависимости со `scope` compile (по умолчанию) и runtime; `test`, `provided`, `system` и `<optional>true</optional>` пропускаются (они не транзитивны). Зависимость без определимой версии или с диапазоном версий пропускается с предупреждением.
- Ошибки: нет POM корневого пакета (HTTP 404), репозиторий недоступен, некорректный XML. У дочерних пакетов такая ошибка - предупреждение, пакет считается листом.
- Проверялось на макете репозитория `tests/mock_repo` (его раздаёт `python3 -m http.server`), реальный Maven Central из среды разработки недоступен.

Демонстрация (`bash tests/test_stage2.sh`, макет репозитория поднимается скриптом на порту 8765):

```
$ python3 src/main.py tests/configs/stage2_app.json --stage 2
Прямые зависимости org.example:app:1.0:
  org.example:lib-a:2.0
  org.example:lib-b:1.5
  org.example:ghost:1.0
Предупреждение: org.example:unknown-version: версия не определена, пропущена
[код возврата: 0]

$ python3 src/main.py tests/configs/stage2_lib_b.json --stage 2
Прямые зависимости org.example:lib-b:1.5:
  org.example:lib-c:1.0
  org.example:lib-a:2.0
[код возврата: 0]

$ python3 src/main.py tests/configs/stage2_leaf.json --stage 2
Прямые зависимости org.example:lib-c:1.0:
  (нет)
[код возврата: 0]

$ python3 src/main.py tests/configs/stage2_cycle.json --stage 2
Прямые зависимости org.example:cyc-x:1.0:
  org.example:cyc-y:1.0
[код возврата: 0]

$ python3 src/main.py tests/configs/stage2_err_missing_pom.json --stage 2
Ошибка: POM недоступен (HTTP 404): http://localhost:8765/org/example/ghost/1.0/ghost-1.0.pom
[код возврата: 1]

$ python3 src/main.py tests/configs/stage2_err_broken_pom.json --stage 2
Ошибка: некорректный XML в POM: mismatched tag: line 1, column 17
[код возврата: 1]

$ python3 src/main.py tests/configs/stage2_err_no_server.json --stage 2
Ошибка: репозиторий недоступен ([Errno 111] Connection refused): http://localhost:9/org/example/app/1.0/app-1.0.pom
[код возврата: 1]
```

Этап 3. Основные операции

- Граф строится **обходом в ширину без рекурсии** (`src/graph.py`, `build_graph`): очередь `collections.deque`, множество посещённых узлов. Узлы на глубине `max_depth` попадают в граф, но не раскрываются (глубина 0 - только корень; в выводе у таких узлов «(нет)», хотя в репозитории зависимости могут быть).
- **Циклы**: уже посещённый узел повторно не обходится, поэтому циклы и самозависимость (`X: X`) не приводят к зацикливанию; ребро в уже известный узел остаётся в графе. Наличие цикла определяется алгоритмом Кана (тоже без рекурсии) и выводится строкой «Циклические зависимости».
- **Режим тестирования** (`test_mode: true`): `repository` - путь к файлу вида `A: B C` (по строке на пакет, имена - большие латинские буквы; пустые строки и строки с `#` пропускаются). Ошибки файла: неверная строка, неверное имя, повторное описание, отсутствие корневого пакета. Примеры - `tests/data/graph_*.txt`.
- Для Maven-репозитория граф строится так же; узел - `groupId:artifactId:version`.

Демонстрация (`bash tests/test_stage3.sh`; наборы: обычный граф с глубиной 3/1/0, цикл, самозависимость, цепочка с ограничением глубины, ошибки файла, Maven с предупреждениями и Maven-цикл):

```
$ python3 src/main.py tests/configs/stage3_basic_d3.json --stage 3
Граф зависимостей A (глубина не больше 3):
  A -> B, C
  B -> D
  C -> D, E
  D -> (нет)
  E -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage3_basic_d1.json --stage 3
Граф зависимостей A (глубина не больше 1):
  A -> B, C
  B -> (нет)
  C -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage3_basic_d0.json --stage 3
Граф зависимостей A (глубина не больше 0):
  A -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage3_cycle.json --stage 3
Граф зависимостей A (глубина не больше 10):
  A -> B
  B -> C
  C -> A, D
  D -> (нет)
Циклические зависимости: да
[код возврата: 0]

$ python3 src/main.py tests/configs/stage3_self.json --stage 3
Граф зависимостей X (глубина не больше 5):
  X -> X, Y
  Y -> X
Циклические зависимости: да
[код возврата: 0]

$ python3 src/main.py tests/configs/stage3_chain_d2.json --stage 3
Граф зависимостей A (глубина не больше 2):
  A -> B
  B -> C
  C -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage3_chain_d9.json --stage 3
Граф зависимостей A (глубина не больше 9):
  A -> B
  B -> C
  C -> D
  D -> E
  E -> F
  F -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage3_err_missing_root.json --stage 3
Ошибка: пакет 'Z' не найден в тестовом репозитории
[код возврата: 1]

$ python3 src/main.py tests/configs/stage3_err_bad_line.json --stage 3
Ошибка: строка 1: ожидается 'ИМЯ: ЗАВИСИМОСТИ', имена - большие латинские буквы
[код возврата: 1]

$ python3 src/main.py tests/configs/stage3_err_bad_name.json --stage 3
Ошибка: строка 2: ожидается 'ИМЯ: ЗАВИСИМОСТИ', имена - большие латинские буквы
[код возврата: 1]

$ python3 src/main.py tests/configs/stage3_err_duplicate.json --stage 3
Ошибка: строка 3: пакет 'A' описан повторно
[код возврата: 1]

$ python3 src/main.py tests/configs/stage3_err_bad_dep.json --stage 3
Ошибка: строка 1: недопустимое имя пакета 'c'
[код возврата: 1]

$ python3 src/main.py tests/configs/stage3_maven_app.json --stage 3
Граф зависимостей org.example:app:1.0 (глубина не больше 3):
  org.example:app:1.0 -> org.example:lib-a:2.0, org.example:lib-b:1.5, org.example:ghost:1.0
  org.example:lib-a:2.0 -> org.example:lib-c:1.0
  org.example:lib-b:1.5 -> org.example:lib-c:1.0, org.example:lib-a:2.0
  org.example:ghost:1.0 -> (нет)
  org.example:lib-c:1.0 -> (нет)
Циклические зависимости: нет
Предупреждение: org.example:unknown-version: версия не определена, пропущена
Предупреждение: org.example:ghost:1.0: POM недоступен (HTTP 404): http://localhost:8765/org/example/ghost/1.0/ghost-1.0.pom
[код возврата: 0]
```

```
$ python3 src/main.py tests/configs/stage3_maven_cycle.json --stage 3
Граф зависимостей org.example:cyc-x:1.0 (глубина не больше 10):
  org.example:cyc-x:1.0 -> org.example:cyc-y:1.0
  org.example:cyc-y:1.0 -> org.example:cyc-x:1.0
Циклические зависимости: да
[код возврата: 0]
```

Этап 4. Дополнительные операции

- Режим обратных зависимостей включается параметром `"reverse": true` (только для тестового репозитория). Обратные зависимости пакета - это пакеты, которые зависят от него (в том числе транзитивно). Для этого рёбра графа тестового репозитория обращаются (`graph.invert`), после чего используется **тот же BFS** `build_graph`, что и на этапе 3: с ограничением глубины и защитой от циклов.
- В выводе запись `X -> Y` читается как «от X зависит Y». Узлы на предельной глубине не раскрываются, как и в прямом режиме.
- Для Maven-репозитория обратные зависимости получить нельзя: в репозитории нет индекса «кто зависит от пакета», поэтому `reverse: true` с `test_mode: false` - ошибка конфигурации.
- Демонстрация на разных случаях: пакет в середине графа (`E`, `D`), на предельной глубине, корень без зависимых (`A`), цикл, граф с циклом через несколько путей (`G`), отсутствующий пакет.

Демонстрация (`bash tests/test_stage4.sh`):

```
$ python3 src/main.py tests/configs/stage3_basic_d3.json --stage 3
Граф зависимостей A (глубина не больше 3):
  A -> B, C
  B -> D
  C -> D, E
  D -> (нет)
  E -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage4_basic_e.json --stage 4
Обратные зависимости E - пакеты, которые от него зависят (глубина не больше 5).
Запись 'X -> Y' читается так: от X зависит Y:
  E -> C
  C -> A
  A -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage4_basic_d.json --stage 4
Обратные зависимости D - пакеты, которые от него зависят (глубина не больше 5).
Запись 'X -> Y' читается так: от X зависит Y:
  D -> B, C
  B -> A
  C -> A
  A -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage4_basic_d_depth1.json --stage 4
Обратные зависимости D - пакеты, которые от него зависят (глубина не больше 1).
Запись 'X -> Y' читается так: от X зависит Y:
  D -> B, C
  B -> (нет)
  C -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage4_basic_root.json --stage 4
Обратные зависимости A - пакеты, которые от него зависят (глубина не больше 5).
Запись 'X -> Y' читается так: от X зависит Y:
  A -> (нет)
Циклические зависимости: нет
[код возврата: 0]

$ python3 src/main.py tests/configs/stage4_cycle_c.json --stage 4
Обратные зависимости C - пакеты, которые от него зависят (глубина не больше 10).
Запись 'X -> Y' читается так: от X зависит Y:
  C -> B
  B -> A
  A -> C
Циклические зависимости: да
[код возврата: 0]

$ python3 src/main.py tests/configs/stage4_demo_g.json --stage 4
Обратные зависимости G - пакеты, которые от него зависят (глубина не больше 10).
Запись 'X -> Y' читается так: от X зависит Y:
  G -> D, E
  D -> B
  E -> B, C
  B -> A
  C -> A
  A -> F
  F -> C
Циклические зависимости: да
[код возврата: 0]

$ python3 src/main.py tests/configs/stage4_err_missing.json --stage 4
Ошибка: пакет 'Z' не найден в тестовом репозитории
[код возврата: 1]

$ python3 src/main.py tests/configs/errors/reverse_in_maven.json --stage 4
Ошибка конфигурации: reverse: обратные зависимости доступны только для тестового репозитория (test_mode)
[код возврата: 2]
```

Этап 5. Визуализация

- Граф выводится на языке диаграмм **D2** (`render.d2`): строки `"A" -> "B"` (стрелка от зависимого к зависимости); имена в кавычках, поэтому подходят и `A`, и `org.example:app:1.0`. Узел без рёбер объявляется отдельной строкой. В режиме обратных зависимостей стрелки идут так же - от зависимого пакета к зависимости.
- Если `ascii_tree: true`, дополнительно печатается ASCII-дерево (`render.ascii_tree`, обход со стеком, без рекурсии). Узел, уже раскрытый выше, помечается `(повтор)` и не раскрывается повторно; возврат к предку - `(цикл)`.
- Описание D2 можно отрисовать в картинку утилитой `d2` (`d2 graph.d2 graph.svg`); в проекте она не нужна.
- Примеры для **трёх различных пакетов** тестового репозитория `tests/data/graph_demo.txt` (A, C, D; у A и C есть цикл через F, у D - один зависимый), обратный режим (G), ограничение глубины и Maven-макет (`org.example:app`).

Демонстрация (`bash tests/test_stage5.sh`):

```
$ python3 src/main.py tests/configs/stage5_pkg_a.json
Описание графа на языке D2:
"A" -> "B"
"A" -> "C"
"B" -> "D"
"B" -> "E"
"C" -> "E"
"C" -> "F"
"D" -> "G"
"E" -> "G"
"F" -> "A"
ASCII-дерево зависимостей:
A
├── B
│   ├── D
│   │   └── G
│   └── E
│       └── G
└── C
    ├── E (повтор)
    └── F
        └── A (цикл)
[код возврата: 0]

$ python3 src/main.py tests/configs/stage5_pkg_c.json
Описание графа на языке D2:
"C" -> "E"
"C" -> "F"
"E" -> "G"
"F" -> "A"
"A" -> "B"
"A" -> "C"
"B" -> "D"
"B" -> "E"
"D" -> "G"
ASCII-дерево зависимостей:
C
├── E
│   └── G
└── F
    └── A
        ├── B
        │   ├── D
        │   │   └── G
        │   └── E (повтор)
        └── C (цикл)
[код возврата: 0]

$ python3 src/main.py tests/configs/stage5_pkg_d.json
Описание графа на языке D2:
"D" -> "G"
ASCII-дерево зависимостей:
D
└── G
[код возврата: 0]

$ python3 src/main.py tests/configs/stage5_no_tree.json
Описание графа на языке D2:
"A" -> "B"
"A" -> "C"
"B" -> "D"
"B" -> "E"
"C" -> "E"
"C" -> "F"
"D" -> "G"
"E" -> "G"
"F" -> "A"
[код возврата: 0]

$ python3 src/main.py tests/configs/stage5_depth2.json
Описание графа на языке D2:
"A" -> "B"
"A" -> "C"
"B" -> "D"
"B" -> "E"
"C" -> "E"
"C" -> "F"
ASCII-дерево зависимостей:
A
├── B
│   ├── D
│   └── E
└── C
    ├── E
    └── F
[код возврата: 0]

$ python3 src/main.py tests/configs/stage5_reverse_g.json
Описание графа на языке D2:
"D" -> "G"
"E" -> "G"
"B" -> "D"
"B" -> "E"
"C" -> "E"
"A" -> "B"
```

```
"A" -> "C"
"F" -> "A"
"C" -> "F"
ASCII-дерево зависимостей:
G
├── D
│   └── B
│       └── A
│           └── F
│               └── C
│                   └── A (цикл)
└── E
    ├── B (повтор)
    └── C (повтор)
[код возврата: 0]

$ python3 src/main.py tests/configs/stage5_maven_app.json
Описание графа на языке D2:
"org.example:app:1.0" -> "org.example:lib-a:2.0"
"org.example:app:1.0" -> "org.example:lib-b:1.5"
"org.example:app:1.0" -> "org.example:ghost:1.0"
"org.example:lib-a:2.0" -> "org.example:lib-c:1.0"
"org.example:lib-b:1.5" -> "org.example:lib-c:1.0"
"org.example:lib-b:1.5" -> "org.example:lib-a:2.0"
ASCII-дерево зависимостей:
org.example:app:1.0
├── org.example:lib-a:2.0
│   └── org.example:lib-c:1.0
├── org.example:lib-b:1.5
│   ├── org.example:lib-c:1.0
│   └── org.example:lib-a:2.0 (повтор)
└── org.example:ghost:1.0
Предупреждение: org.example:unknown-version: версия не определена, пропущена
Предупреждение: org.example:ghost:1.0: POM недоступен (HTTP 404): http://localhost:8765/org/example/ghost/1.0/ghost-1.0.pom
[код возврата: 0]
```

Сравнение со штатным инструментом (Maven)

Штатный инструмент Maven - `mvn dependency:tree`. Получить его вывод для пакета можно командой `bash tests/compare_mvn.sh groupId:artifactId:version` (нужны Maven и интернет). Для сравнения с нашим инструментом подготовлены конфигурации реальных пакетов `tests/configs/real_*.json` (`commons-text`, `guava`, `slf4j-api`) с репозиторием `https://repo1.maven.org/maven2`:

```
python3 src/main.py tests/configs/real_commons_text.json
bash tests/compare_mvn.sh org.apache.commons:commons-text:1.10.0
```

Среда, в которой делалась работа, не имела доступа в интернет и к Maven, поэтому сами реальные пакеты здесь не запускались; логика проверена на макете репозитория. Результаты сравнения на реальных пакетах надо добавить после запуска двух команд выше (вставить оба вывода сюда).

Расхождения, которые ожидаются по устройству инструментов (и которые надо объяснять, если они появятся):

Причина	Что делает Maven	Что делает наш инструмент
Конфликт версий	оставляет одну версию («ближайшая побеждает»), остальные помечает `omitted for conflict`	показывает все версии как отдельные узлы
Дубликаты	повторный узел помечает `omitted for duplicate` и не раскрывает	помечает `(повтор)` и не раскрывает
Области видимости	в дереве проекта показывает и `test`, и `provided` у прямых зависимостей	берёт только `compile` и `runtime`
`optional`	у транзитивных зависимостей отбрасывает опциональные	отбрасывает везде
`exclusions`	исключает перечисленные пакеты вместе с их поддеревьями	не учитывает исключения
BOM (`import` в `dependencyManagement`), диапазоны версий `[1.0,2.0)`, профили, relocation	поддерживает	не поддерживает: такие зависимости пропускаются с предупреждением
Корень дерева	корень - проект-пустышка `local:probe`, нужный пакет на уровне 1	корень - сам пакет
Глубина	печатает всё дерево	ограничивает `max_depth`

Часть расхождений не требует реального репозитория и видна на макете: у `org.example:app` зависимость `lib-b` тянет `lib-a` и `lib-c`, которые уже есть в графе, - у нас они показаны как отдельные рёбра, а `mvn dependency:tree` пометил бы их как `omitted for duplicate`.

Известные ограничения

- Реальный Maven Central в среде разработки не проверялся (см. выше): разбор POM рассчитан на типичные случаи.
- Не поддерживаются: импорт BOM, диапазоны версий, профили, исключения `exclusions`, разрешение конфликтов версий.